INFORME TECNICO

API REST - Sistema de Gestion de Gimnasio

Proyecto:	GimnasioDB API
Framework :	C# .NET 8 – Entity Framework Core – JWT
Base de datos:	SQL Server (GimnasioDB)
Repositorio	github.com/ArturoU5/ExmFinal
Integrantes	- Guillermo Arturo Ugaz Montesinos - Josmel Montoya Paiva - Bryan Thomas Szott Sangama - Angello Andre Ramírez Del Aguila - Omar Alexander Córdova Pintado - Marco Antonio Suarez Siesquen

1. Introduccion

El presente informe describe la arquitectura, configuracion, endpoints, validaciones y modelo de seguridad de la API REST desarrollada para la gestion integral de un gimnasio. La solucion fue construida sobre .NET 10 con ASP.NET Core y expone recursos para administrar socios, entrenadores, rutinas, ejercicios, membresias y asistencias, protegidos mediante autenticacion JWT y control de acceso basado en roles.

El objetivo de este documento es servir como referencia tecnica para desarrolladores que deseen instalar, configurar, consumir o extender la API.

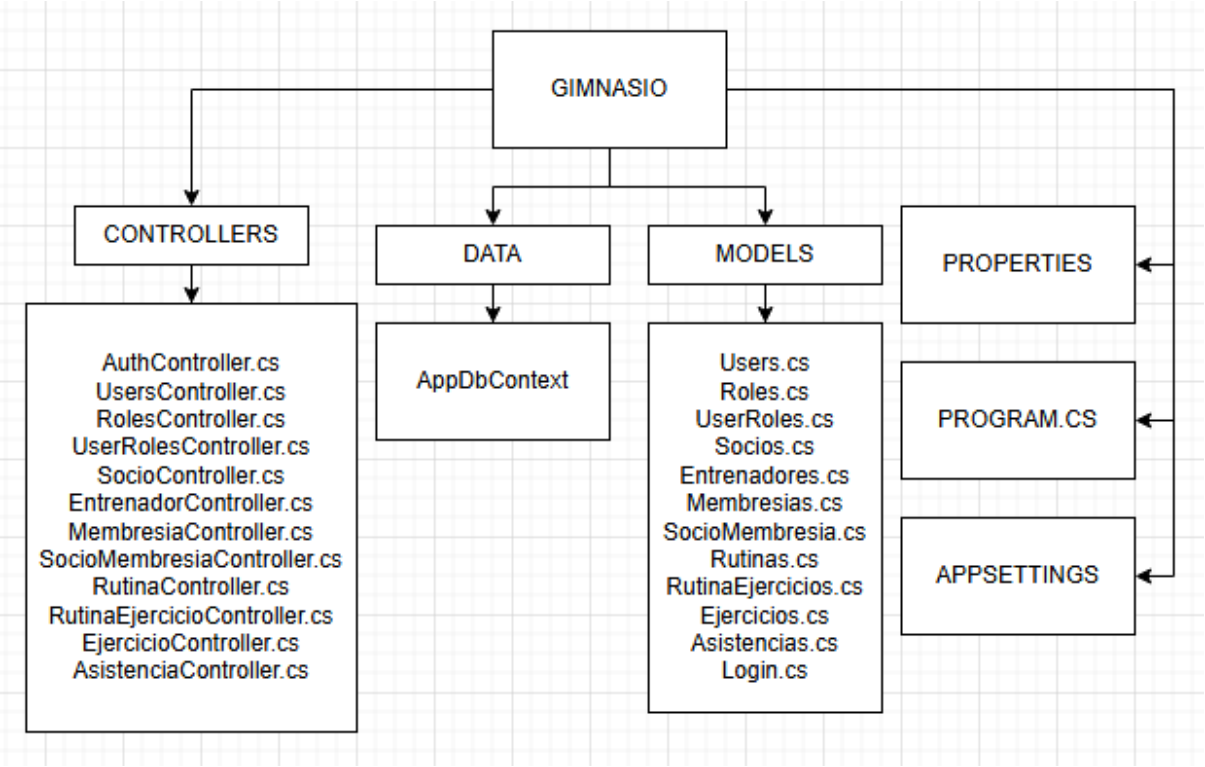
2. Tecnologias Implementadas

La siguiente tabla resume el stack tecnologico utilizado en el proyecto:

Tecnologia	Version	Proposito
.NET	10.0	Framework principal de la aplicacion
ASP.NET Core	10.0	Motor de la Web API RESTful
Entity Framework Core	10.0	ORM para acceso y mapeo de datos
SQL Server	-	Motor de base de datos relacional
JWT / JwtBearer	10.0	Autenticacion stateless con tokens
Swashbuckle / Swagger	10.1.5	Documentacion interactiva de la API

3. Arquitectura del Sistema

El proyecto sigue una arquitectura en capas dentro de un unico proyecto ASP.NET Core Web API. Las responsabilidades estan distribuidas en tres capas principales:



3.1 Estructura de directorios

```
ExmFinal/
  Controllers/   <- Controladores HTTP (12 controladores)
  Models/        <- Entidades del dominio con Data Annotations
  Data/          <- AppDbContext (Entity Framework Core)
  Program.cs     <- Configuracion de servicios y pipeline
  appsettings.json <- Cadena de conexion y clave JWT
```

3.2 Flujo de una solicitud

Cada solicitud HTTP pasa por el siguiente pipeline antes de llegar al controlador:

> Cliente HTTP --> [Middleware: Authentication / Authorization] --> Controller --> AppDbContext (EF Core) --> SQL Server

El middleware de autenticacion valida el token JWT y extrae el rol del usuario. El middleware de autorizacion verifica que el rol tenga acceso al endpoint solicitado. Si cualquiera de estas verificaciones falla, la solicitud es rechazada antes de llegar al controlador.

3.3 Modelo de datos

El sistema gestiona las siguientes entidades principales en la base de datos:

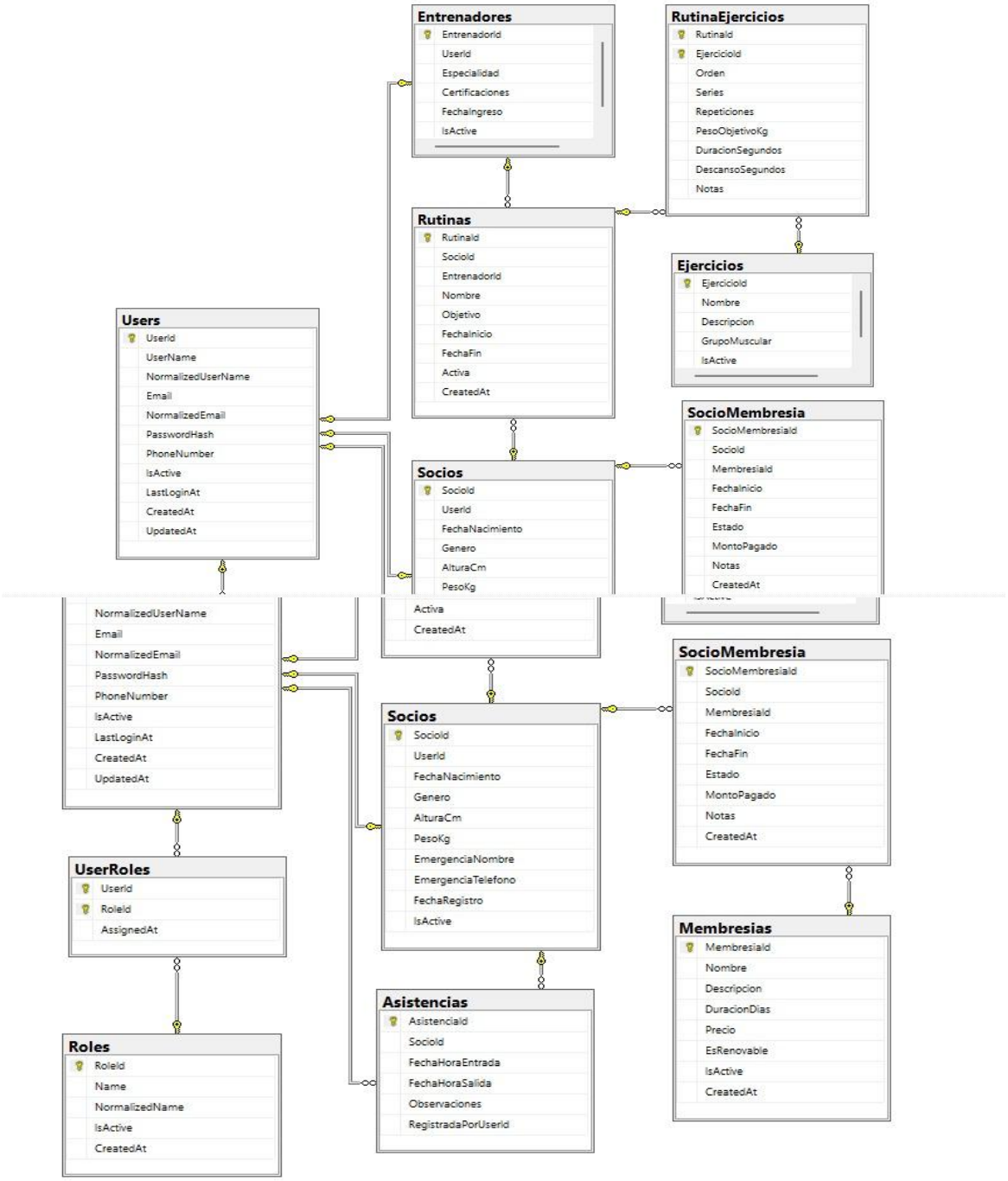


Tabla SQL Server	Clase C# (Model)	Controller
dbo.Users	User	UserController
dbo.Roles	Role	RoleController
dbo.UserRoles	UserRole	UserRoleController
dbo.Socios	Socio	SocioController
dbo.Entrenadores	Entrenador	EntrenadorController
dbo.Membresias	Membresia	MembresiaController
dbo.SocioMembresia	SocioMembresia	SocioMembresiasController
dbo.Asistencias	Asistencia	AsistenciaController
dbo.Rutinas	Rutina	RutinaController
dbo.Ejercicios	Ejercicio	EjercicioController

4. Instalacion y Configuracion

4.1 Requisitos previos

- .NET 10 SDK (https://dotnet.microsoft.com/download)
- SQL Server local o remoto
- Git
- dotnet-ef CLI tool

4.2 Clonar el repositorio

```
git clone https://github.com/ArturoU5/ExmFinal.git
cd ExmFinal
```

4.3 Configurar appsettings.json

Editar el archivo appsettings.json con la cadena de conexion al servidor SQL y una clave JWT segura (minimo 32 caracteres):

```
"ConnectionStrings": {
  "DefaultConnection": "Server=localhost;Database=GimnasioDB;
   Trusted_Connection=True;TrustServerCertificate=True;"
},
"Jwt": {
  "Key": "TuClaveSecretaDe32CaracteresComoMinimo2024!"
}
```

> IMPORTANTE: La clave JWT debe tener al menos 32 caracteres para el algoritmo HMAC-SHA256. Nunca compartir esta clave en repositorios publicos.

4.4 Restaurar dependencias y aplicar migraciones

```
dotnet restore
dotnet tool install --global dotnet-ef   (si no esta instalado)
dotnet ef migrations add InitialCreate
dotnet ef database update
```

4.5 Ejecutar la aplicacion

```
dotnet run
```

La API quedara disponible en los siguientes endpoints base:

- Swagger UI: http://localhost:5000/swagger
- http://localhost:5027

5. Autenticacion

La API utiliza JSON Web Tokens (JWT) para autenticar todas las solicitudes. El token tiene una vigencia de 2 horas y debe enviarse en el encabezado Authorization de cada peticion.

5.1 Obtener un token

Realizar una solicitud POST al endpoint de login:

```
POST /api/auth/login
Content-Type: application/json

{ "nombre": "admin", "password": "hash_del_password" }
```

Respuesta exitosa (200 OK):

```
{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." }
```

5.2 Usar el token

Incluir el token en el encabezado de cada solicitud protegida:

```
Authorization: Bearer <token>
```

Si el token es invalido o ha expirado, la API respondera con 401 Unauthorized. Si el rol del usuario no tiene acceso al recurso, respondera con 403 Forbidden.

6. Roles y Control de Acceso

El sistema implementa autorizacion basada en roles (RBAC). Existen tres roles predefinidos:

- ADMIN: Acceso completo a todos los endpoints del sistema.
- ENTRENADOR: Gestion de rutinas, ejercicios y asistencias. Puede ver su propio perfil de entrenador.
- SOCIO: Acceso de solo lectura a su propio perfil, sus rutinas asignadas y sus membresias.

La siguiente matriz resume los permisos por modulo:

Modulo	ADMIN	ENTRENADOR	SOCIO
Usuarios	CRUD completo	No	No
Roles	CRUD completo	No	No
Asignacion de roles	CRUD completo	No	No
Socios (listado)	CRUD completo	Solo lectura	No
Socios (perfil propio)	CRUD completo	Solo lectura	Solo el suyo
Entrenadores (listado)	CRUD completo	No	No
Entrenadores (perfil)	CRUD completo	Solo el suyo	No
Membresias	CRUD completo	Solo lectura	Solo lectura
Socio-Membresia	CRUD completo	No	Solo la suya
Rutinas (listado)	CRUD completo	CRUD completo	No
Rutinas (detalle)	CRUD completo	CRUD completo	Solo la suya
Ejercicios	CRUD completo	Solo lectura	Solo lectura
Rutina-Ejercicio	CRUD completo	CRUD completo	Solo lectura
Asistencias	CRUD completo	Crear/Editar/Leer	Solo lectura

> Restriccion adicional: Los roles SOCIO y ENTRENADOR solo pueden acceder a sus propios recursos (perfil, rutinas, membresias). Intentar acceder a recursos de otros usuarios retorna 403 Forbidden.

7. Endpoints Documentados

7.1 Autenticacion — /api/auth

Método	Endpoint	Descripción	Roles permitidos
POST	`/api/auth/login`	Genera un token JWT valido por 2 horas	Publico

7.2 Usuarios — /api/users

Método	Endpoint	Descripción	Roles permitidos
GET	`/api/users`	Listar todos los usuarios activos	ADMIN
GET	`/api/users/{id}`	Obtener un usuario por ID	ADMIN
POST	`/api/users`	Crear un nuevo usuario	ADMIN
PUT	`/api/users/{id}`	Actualizar datos de un usuario	ADMIN
DELETE	`/api/users/{id}`	Eliminar un usuario	ADMIN

Body esperado (POST / PUT)

```
{ "userName": "johndoe", "normalizedUserName": "JOHNDOE",
  "email": "john@example.com", "normalizedEmail": "JOHN@EXAMPLE.COM",
  "passwordHash": "hash_password", "phoneNumber": "999999999",
  "isActive": true }
```

7.3 Roles — /api/roles

Método	Endpoint	Descripción	Roles permitidos
GET	`/api/roles`	Listar roles activos	ADMIN
GET	`/api/roles/{id}`	Ver un rol por ID	ADMIN
POST	`/api/roles`	Crear un nuevo rol	ADMIN
PUT	`/api/roles/{id}`	Actualizar un rol	ADMIN
DELETE	`/api/roles/{id}`	Eliminar un rol	ADMIN

7.4 Asignacion de Roles — /api/user-roles

Método	Endpoint	Descripción	Roles permitidos
GET	`/api/user-roles/{userId}`	Ver roles de un usuario	ADMIN
POST	`/api/user-roles`	Asignar rol a un usuario	ADMIN
DELETE	`/api/user-roles/{userId}/{roleId}`	Revocar un rol de usuario	ADMIN

7.5 Socios — /api/socio

Método	Endpoint	Descripción	Roles permitidos
GET	`/api/socio`	Listar socios activos	ADMIN, ENTRENADOR
GET	`/api/socio/{id}`	Ver perfil de un socio	ADMIN, ENTRENADOR, SOCIO*
POST	`/api/socio`	Registrar un nuevo socio	ADMIN
PUT	`/api/socio/{id}`	Actualizar datos del socio	ADMIN, SOCIO*
DELETE	`/api/socio/{id}`	Eliminar un socio	ADMIN

> (*) SOCIO solo puede consultar y editar su propio perfil. El campo isActive solo puede modificarlo ADMIN.

7.6 Entrenadores — /api/entrenador

Método	Endpoint	Descripción	Roles permitidos
GET	`/api/entrenador`	Listar entrenadores activos	ADMIN
GET	`/api/entrenador/{id}`	Ver perfil de un entrenador	ADMIN, ENTRENADOR*
POST	`/api/entrenador`	Registrar un entrenador	ADMIN
PUT	`/api/entrenador/{id}`	Actualizar datos del entrenador	ADMIN
DELETE	`/api/entrenador/{id}`	Eliminar un entrenador	ADMIN

> (*) ENTRENADOR solo puede consultar su propio perfil de entrenador.

7.7 Membresias — /api/membresia

Método	Endpoint	Descripción	Roles permitidos
GET	`/api/membresia`	Listar membresias activas	ADMIN, ENTRENADOR, SOCIO
GET	`/api/membresia/{id}`	Ver una membresia por ID	ADMIN, ENTRENADOR, SOCIO
POST	`/api/membresia`	Crear una nueva membresia	ADMIN
PUT	`/api/membresia/{id}`	Actualizar una membresia	ADMIN
DELETE	`/api/membresia/{id}`	Eliminar una membresia	ADMIN

7.8 Socio-Membresia — /api/socio-membresia

Método	Endpoint	Descripción	Roles permitidos
GET	`/api/socio-membresia/{socioId}`	Ver membresias de un socio	ADMIN, SOCIO*
GET	`/api/socio-membresia/{socioId}/{smId}`	Ver una membresia especifica	ADMIN, SOCIO*
POST	`/api/socio-membresia`	Asignar membresia a un socio	ADMIN
PUT	`/api/socio-membresia/{socioMembresiaId}`	Actualizar membresia del socio	ADMIN
DELETE	`/api/socio-membresia/{socioMembresiaId}`	Eliminar membresia del socio	ADMIN

> (*) SOCIO solo puede consultar sus propias membresias.

7.9 Rutinas — /api/rutina

Método	Endpoint	Descripción	Roles permitidos
GET	`/api/rutina`	Listar rutinas activas	ADMIN, ENTRENADOR

Método	Endpoint	Descripción	Roles permitidos
GET	`/api/rutina/{id}`	Ver una rutina especifica	ADMIN, ENTRENADOR, SOCIO*
POST	`/api/rutina`	Crear una nueva rutina	ADMIN, ENTRENADOR
PUT	`/api/rutina/{id}`	Actualizar una rutina	ADMIN, ENTRENADOR
DELETE	`/api/rutina/{id}`	Eliminar una rutina	ADMIN, ENTRENADOR

> (*) SOCIO solo puede consultar rutinas que le han sido asignadas.

7.10 Ejercicios — /api/ejercicio

Método	Endpoint	Descripción	Roles permitidos
GET	`/api/ejercicio`	Listar ejercicios activos	ADMIN, ENTRENADOR, SOCIO
GET	`/api/ejercicio/{id}`	Ver un ejercicio por ID	ADMIN, ENTRENADOR, SOCIO
POST	`/api/ejercicio`	Crear un ejercicio	ADMIN
PUT	`/api/ejercicio/{id}`	Actualizar un ejercicio	ADMIN
DELETE	`/api/ejercicio/{id}`	Eliminar un ejercicio	ADMIN

7.11 Rutina-Ejercicio — /api/rutina-ejercicio

Método	Endpoint	Descripción	Roles permitidos
GET	`/api/rutina-ejercicio/{rutinaId}`	Ver ejercicios de una rutina	ADMIN, ENTRENADOR, SOCIO
POST	`/api/rutina-ejercicio`	Agregar ejercicio a una rutina	ADMIN, ENTRENADOR
PUT	`/api/rutina-ejercicio/{rutinaId}/{ejId}`	Actualizar ejercicio en rutina	ADMIN, ENTRENADOR
DELETE	`/api/rutina-ejercicio/{rutinaId}/{ejId}`	Quitar ejercicio de una rutina	ADMIN, ENTRENADOR

7.12 Asistencias — /api/asistencia

Método	Endpoint	Descripción	Roles permitidos
GET	`/api/asistencia`	Listar todas las asistencias	ADMIN, ENTRENADOR, SOCIO
GET	`/api/asistencia/{id}`	Ver una asistencia por ID	ADMIN, ENTRENADOR, SOCIO
POST	`/api/asistencia`	Registrar una asistencia	ADMIN, ENTRENADOR

Método	Endpoint	Descripción	Roles permitidos
PUT	`/api/asistencia/{id}`	Actualizar una asistencia	ADMIN, ENTRENADOR
DELETE	`/api/asistencia/{id}`	Eliminar una asistencia	ADMIN

8. Validaciones y Manejo de Errores

8.1 Validaciones de modelo (Data Annotations)

Todos los modelos aplican validaciones automaticas mediante anotaciones de datos de .NET. Si el cuerpo del request no las satisface, la API retorna 400 Bad Request con el detalle de cada campo invalido. Las principales restricciones son:

- [Required]: Campo obligatorio. Retorna error si no se incluye en el cuerpo.
- [MaxLength(n)]: Longitud maxima de cadena de texto.
- [Precision(p, s)]: Precision y escala para campos decimales (precios, pesos, medidas).

8.2 Validaciones de negocio por modulo

Modulo	Regla de negocio	HTTP
Usuarios	UserName y Email deben ser unicos en el sistema	400
Roles	Name del rol debe ser unico	400
UserRoles	El usuario y el rol referenciados deben existir	400
UserRoles	Un usuario no puede tener el mismo rol asignado dos veces	400
Socios	El UserId referenciado debe existir	400
Socios	Un usuario no puede ser registrado como socio mas de una vez	400
Entrenadores	El UserId referenciado debe existir	400
Entrenadores	Un usuario no puede ser registrado como entrenador mas de una vez	400
Membresias	Precio debe ser mayor a 0	400
Membresias	DuracionDias debe ser mayor a 0	400
SocioMembresia	SocioId y MembresiaId deben existir	400
SocioMembresia	FechaFin debe ser posterior a FechaInicio	400
SocioMembresia	MontoPagado no puede ser negativo	400
Rutinas	SocioId y EntrenadorId deben existir	400
Rutinas	Si se proporciona FechaFin, debe ser posterior a FechaInicio	400
RutinaEjercicio	RutinaId y EjercicioId deben existir	400
Asistencias	SocioId y RegistradaPorUserId deben existir	400
Asistencias	FechaHoraSalida debe ser posterior a FechaHoraEntrada	400

8.3 Codigos de respuesta HTTP

Codigo HTTP	Significado	Cuando ocurre
200 OK	Solicitud exitosa	GET o PUT exitoso
201 Created	Recurso creado exitosamente	POST exitoso
204 No Content	Operacion exitosa sin cuerpo	DELETE exitoso
400 Bad Request	Datos invalidos o regla de negocio violada	Validaciones fallidas
401 Unauthorized	Token ausente o invalido	Sin autenticacion
403 Forbidden	Rol insuficiente o recurso ajeno	Acceso denegado
404 Not Found	Recurso no encontrado	ID inexistente

8.4 Formato de respuesta de error

Los errores de validacion de negocio retornan un objeto JSON con la siguiente estructura estandar:

```
HTTP 400 Bad Request
{
  "mensaje": "Error de validacion",
  "detalle": "La fecha y hora de salida debe ser posterior a la de entrada."
}
```

Los errores de base de datos o excepciones internas incluyen el mensaje del inner exception para facilitar el diagnostico:

```
HTTP 400 Bad Request
{
  "mensaje": "Error al registrar la asistencia",
  "detalle": "Cannot insert duplicate key in object dbo.Asistencias..."
}
```

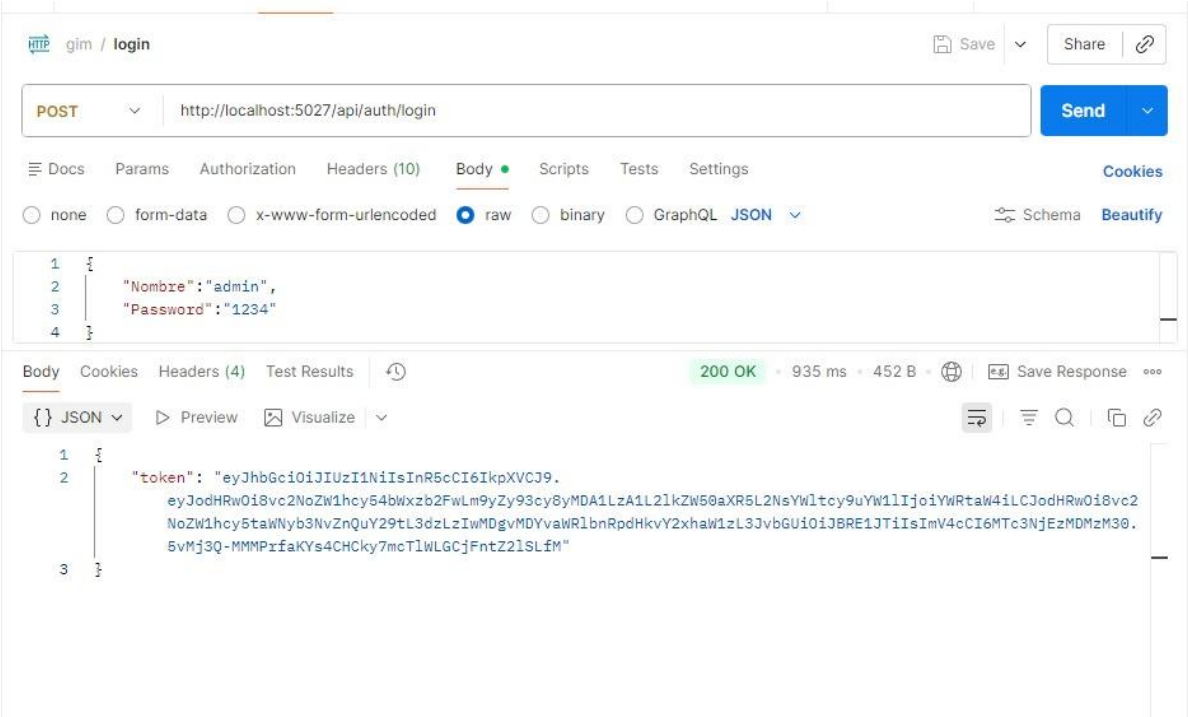
9. Consideraciones de Seguridad

A continuacion se listan buenas practicas recomendadas para entornos de produccion:

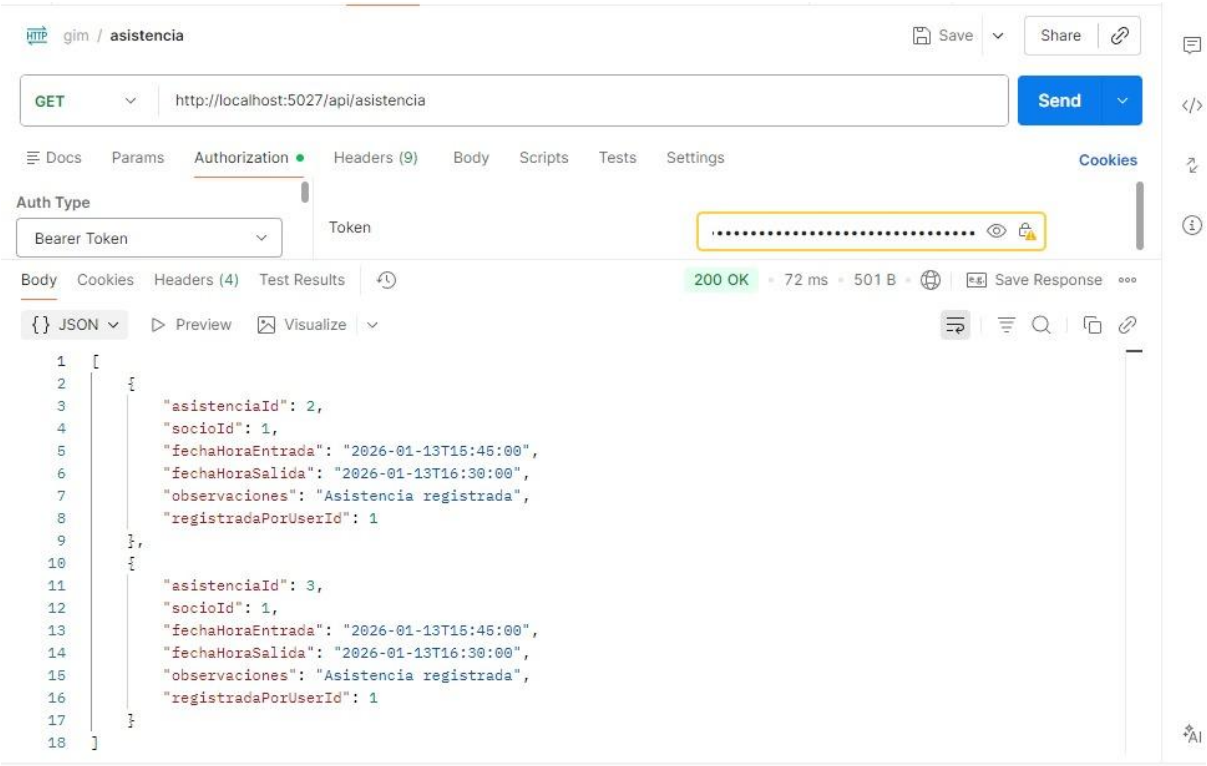
1. Reemplazar la clave JWT por defecto en appsettings.json por una clave larga y aleatoria, gestionada como variable de entorno o secreto del sistema.
2. Implementar hashing de contrasenas (ej. BCrypt) en lugar de almacenar el hash directamente. El campo PasswordHash actualmente no aplica un algoritmo de hashing automatico.
3. Configurar HTTPS en el servidor de produccion y redirigir todo el trafico HTTP.
4. Restringir los origenes CORS permitidos en Program.cs segun el dominio del cliente.
5. Revisar el nivel de detalle expuesto en los mensajes de error antes de desplegar en produccion.
6. Asegurarse de que la cadena de conexion a SQL Server no este expuesta en repositorios publicos.

Evidencias de las pruebas

LOGIN



ASISTENCIA:



gim / asistencia id

Save Share

GET http://localhost:5027/api/asistencia/2 Send

Docs Params Authorization Headers (9) Body Scripts Tests Settings Cookies

Auth Type

Bearer Token

Token

The authorization header will be automatically generated when you

Body Cookies Headers (4) Test Results 200 OK 65 ms 324 B Save Response

JSON Preview Visualize


```
{
    "asistenciaId": 2,
    "socioId": 1,
    "fechaHoraEntrada": "2025-01-13T15:45:00",
    "fechaHoraSalida": "2026-01-13T16:30:00",
    "observaciones": "Asistencia actualizada",
    "registradaPorUserId": 1
}
```


gim / asistencia

Save Share

PUT http://localhost:5027/api/asistencia/2 Send

Docs Params Authorization Headers (11) Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify


```
{
    "SocioId": 1,
    "FechaHoraEntrada": "2025-01-13T15:45:00",
    "FechaHoraSalida": "2026-01-13T16:30:00",
    "Observaciones": "Asistencia actualizada",
    "RegistradaPorUserId": 1
}
```


Body Cookies Headers (4) Test Results 200 OK 22 ms 388 B Save Response

JSON Preview Visualize


```
{
    "mensaje": "Asistencia actualizada correctamente",
    "asistencia": {
        "asistenciaId": 2,
        "socioId": 1,
        "fechaHoraEntrada": "2025-01-13T15:45:00",
        "fechaHoraSalida": "2026-01-13T16:30:00",
        "observaciones": "Asistencia actualizada",
        "registradaPorUserId": 1
    }
}
```

gim / asistencia

Save Share

POST http://localhost:5027/api/asistencia Send

Docs Params Authorization Headers (11) Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2     "SocioId": 1,
3     "FechaHoraEntrada": "2026-01-13T15:45:00",
4     "FechaHoraSalida": "2026-01-13T16:30:00",
5     "Observaciones": "Asistencia registrada",
6     "RegistradaPorUserId": 1
7 }
```

Body Cookies Headers (5) Test Results 201 Created 1.02 s 378 B Save Response

{} JSON Preview Visualize

```
1 {
2     "asistenciaId": 3,
3     "socioId": 1,
4     "fechaHoraEntrada": "2026-01-13T15:45:00",
5     "fechaHoraSalida": "2026-01-13T16:30:00",
6     "observaciones": "Asistencia registrada",
7     "registradaPorUserId": 1
8 }
```

gim / asistencia

Save Share

DELETE http://localhost:5027/api/asistencia/3 Send

Docs Params Authorization Headers (9) Body Scripts Tests Settings Cookies

Auth Type

Be...

Token

The authorization header will be automatically generated when you

Body Cookies Headers (2) Test Results 204 No Content 18 ms 81 B Save Response

Raw Preview Visualize

```
1
```

USER

gim / users

Save Share

GET http://localhost:5027/api/users Send

Docs Params Authorization Headers (11) Body Scripts Tests Settings Cookies

Auth Type

Bearer Token

Token

Body Cookies Headers (4) Test Results 200 OK 111 ms 952 B Save Response

JSON Preview Visualize


```
[
    {
        "userId": 1,
        "userName": "admin",
        "normalizedUserName": "ADMIN",
        "email": "admin@gym.local",
        "normalizedEmail": "ADMIN@GYM.LOCAL",
        "passwordHash": "1234",
        "phoneNumber": "+51 900000001",
        "isActive": true,
        "lastLoginAt": null,
        "createdAt": "2026-04-08T23:09:42",
        "updatedAt": null
    },
    {
        "userId": 2,
        "userName": "entrenador",
        "normalizedUserName": "ENTRENADOR",
```


GET http://localhost:5027/api/users/2 Send

Docs Params Authorization Headers (11) Body Scripts Tests Settings Cookies

Auth Type

Bearer Token

Token

The authorization header will be

Body Cookies Headers (4) Test Results 200 OK 1.07 s 428 B Save Response

JSON Preview Visualize


```
{
    "userId": 2,
    "userName": "entrenador",
    "normalizedUserName": "ENTRENADOR",
    "email": "entrenador@gym.local",
    "normalizedEmail": "ENTRENADOR@GYM.LOCAL",
    "passwordHash": "1234",
    "phoneNumber": "+51 900000002",
    "isActive": true,
    "lastLoginAt": null,
    "createdAt": "2026-04-08T23:09:42",
    "updatedAt": null
}
```

gim / users

Save Share

PUT http://localhost:5027/api/users/11 Send

Docs Params Authorization Headers (11) Body Scripts Tests Settings Cookies

Auth Type

Bearer Token

Token

Body Cookies Headers (4) Test Results 200 OK 74 ms 526 B Save Response

JSON Preview Visualize


```
{
    "mensaje": "Usuario actualizado correctamente",
    "user": {
        "userId": 11,
        "userName": "usuario11actuazliado",
        "normalizedUserName": "USUARIO11ACTUALIZADO",
        "email": "11ACT@example.com",
        "normalizedEmail": "11ACT@EXAMPLE.COM",
        "passwordHash": "hashedpassword123",
        "phoneNumber": "+1234567890",
        "isActive": true,
        "lastLoginAt": "2026-04-13T20:23:35",
        "createdAt": "2026-04-13T15:30:00",
        "updatedAt": null
    }
}
```


POST http://localhost:5027/api/users Send

Docs Params Authorization Headers (11) Body Scripts Tests Settings Cookies

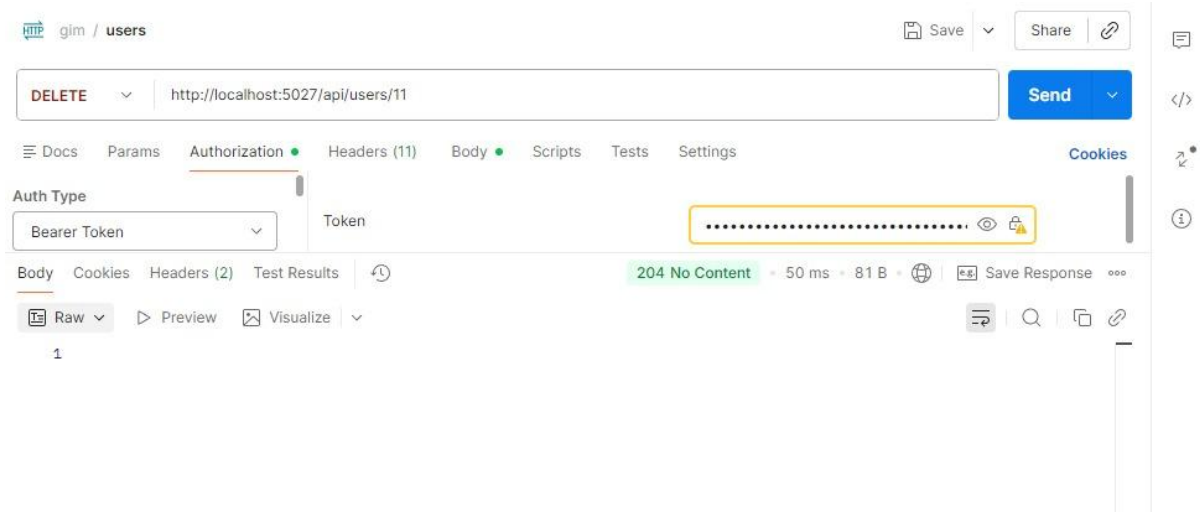
none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify


```
{
    "userName": "usuario11",
    "normalizedUserName": "USUARIO11",
    "email": "11@example.com",
```

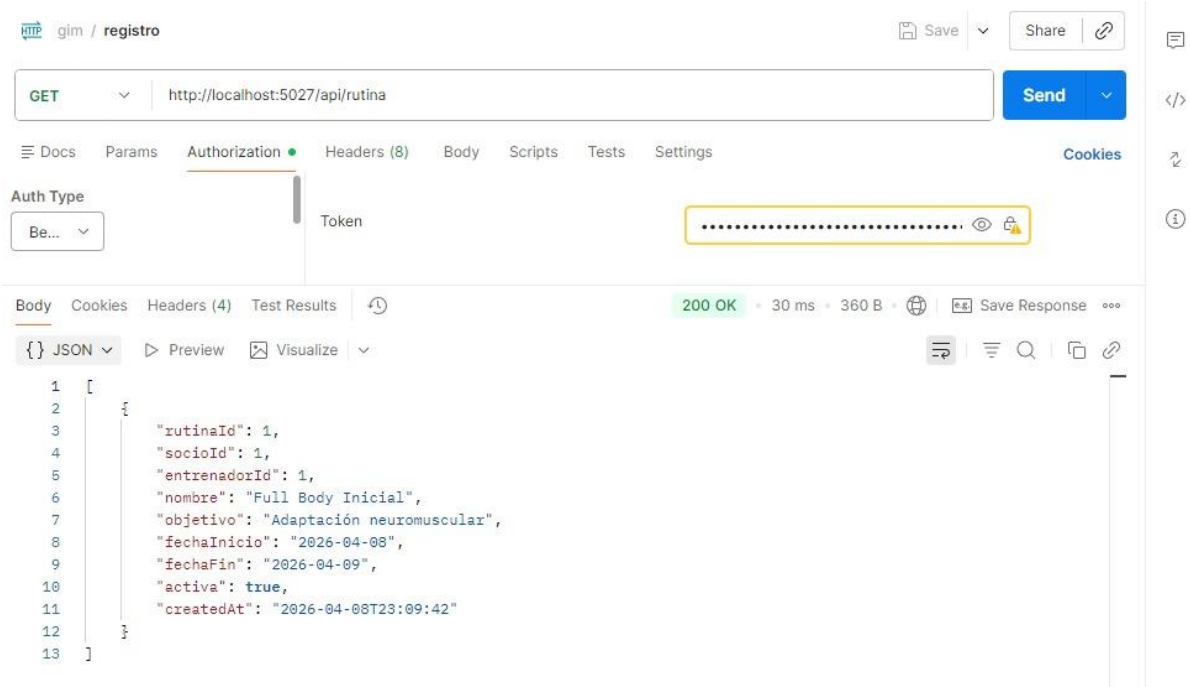

Body Cookies Headers (5) Test Results 201 Created 201 ms 508 B Save Response

JSON Preview Visualize


```
{
    "userId": 11,
    "userName": "usuario11",
    "normalizedUserName": "USUARIO11",
    "email": "11@example.com",
    "normalizedEmail": "11@EXAMPLE.COM",
    "passwordHash": "hashedpassword123",
    "phoneNumber": "+1234567890",
    "isActive": true,
    "lastLoginAt": "2026-04-13T20:23:34.7551084-05:00",
    "createdAt": "2026-04-13T15:30:00",
    "updatedAt": null
}
```


gim / users
Save
Share
DELETE
http://localhost:5027/api/users/11
Send
Docs
Params
Authorization
Headers (11)
Body
Scripts
Tests
Settings
Cookies
Auth Type
Bearer Token
Token
Body
Cookies
Headers (2)
Test Results
204 No Content
50 ms
81 B
Save Response
Raw
Preview
Visualize
1

RUTINAS :


gim / registro
Save
Share
GET
http://localhost:5027/api/rutina
Send
Docs
Params
Authorization
Headers (8)
Body
Scripts
Tests
Settings
Cookies
Auth Type
Be...
Token
Body
Cookies
Headers (4)
Test Results
200 OK
30 ms
360 B
Save Response
JSON
Preview
Visualize
[
{
"rutinaId": 1,
"socioId": 1,
"entrenadorId": 1,
"nombre": "Full Body Inicial",
"objetivo": "Adaptación neuromuscular",
"fechaInicio": "2026-04-08",
"fechaFin": "2026-04-09",
"activa": true,
"createdAt": "2026-04-08T23:09:42"
}
]

gim / rutina id

Save Share

GET http://localhost:5027/api/rutina/1 Send

Docs Params Authorization Headers (8) Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (4) Test Results 200 OK 30 ms 358 B Save Response

{} JSON Preview Visualize


```
{
    "rutinaId": 1,
    "socioId": 1,
    "entrenadorId": 1,
    "nombre": "Full Body Inicial",
    "objetivo": "Adaptación neuromuscular",
    "fechaInicio": "2026-04-08",
    "fechaFin": "2026-04-09",
    "activa": true,
    "createdAt": "2026-04-08T23:09:42"
}
```


gim / registro

Save Share

POST http://localhost:5027/api/rutina Send

Docs Params Authorization Headers (11) Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify


```
{
    "socioId": 1,
    "entrenadorId": 1,
    "nombre": "Full Body avanzado",
    "objetivo": "Adaptación neuromuscular",
```


Body Cookies Headers (5) Test Results 201 Created 147 ms 410 B Save Response

{} JSON Preview Visualize


```
{
    "rutinaId": 2,
    "socioId": 1,
    "entrenadorId": 1,
    "nombre": "Full Body avanzado",
    "objetivo": "Adaptación neuromuscular",
    "fechaInicio": "2026-04-09",
    "fechaFin": "2026-04-10",
    "activa": true,
    "createdAt": "2026-04-09T23:09:42"
}
```

gim / rutina

Save Share

PUT http://localhost:5027/api/rutina/2 Send

Docs Params Authorization Headers (11) Body Scripts Tests Settings Cookies

Auth Type

Bearer Token

Token

ody Cookies Headers (4) Test Results 200 OK 23 ms 427 B Save Response

{} JSON Preview Visualize


```
{
    "mensaje": "Rutina actualizada correctamente",
    "rutina": {
        "rutinaId": 2,
        "socioId": 1,
        "entrenadorId": 1,
        "nombre": "Full Body avanzado actualizado",
        "objetivo": "Adaptación neuromuscular",
        "fechaInicio": "2026-04-09",
        "fechaFin": "2026-04-10",
        "activa": true,
        "createdAt": "2026-04-09T23:09:42"
    }
}
```


gim / delete

Save Share

DELETE http://localhost:5027/api/rutina/2 Send

Docs Params Authorization Headers (8) Body Scripts Tests Settings Cookies

Auth Type

Bearer Token

Token

Body Cookies Headers (2) Test Results 204 No Content 24 ms 81 B Save Response

Raw Preview Visualize